JAVASCRIPT

Semaine 2 – Jour 8

Les tableaux (Arrays)

Cours détaillé • Explications • Exemples • Exercices • QCM

Objectif du jour
Savoir créer un tableau, lire et modifier ses éléments, ajouter et supprimer des valeurs, parcourir son contenu et vérifier si une valeur existe.

Ce document rassemble le contenu du cours du Jour 8 et les exercices à réaliser. Il est conçu pour être relu progressivement : prends le temps de comprendre chaque exemple avant de passer au suivant.

Plan de la semaine 2

La deuxième semaine est consacrée aux données JavaScript et au navigateur :

• **Jour 8 :** tableaux (arrays)

• **Jour 9 :** objets

• **Jour 10 :** méthodes de tableaux

• **Jour 11 :** DOM

• **Jour 12 :** événements

• **Jour 13 :** formulaires

• **Jour 14 :** stockage local (localStorage)

1. Qu'est-ce qu'un tableau ?

Un **tableau** (array) est une structure qui permet de ranger plusieurs valeurs dans une seule variable. Au lieu de créer une variable différente pour chaque produit, on peut regrouper les produits dans un tableau.

```
let produit1 = "Clavier";
let produit2 = "Souris";
let produit3 = "Écran";

// Avec un tableau :
let produits = ["Clavier", "Souris", "Écran"];
```

Le tableau est particulièrement utile lorsqu'on manipule une liste : produits, étudiants, notes, tâches, utilisateurs, etc. Les tableaux seront réutilisés dans les prochains cours.

2. Les indices : comment retrouver une valeur ?

Chaque élément possède une position appelée **indice** (index). Attention : en JavaScript, le premier indice est **0**, et non 1.

```
let fruits = ["Pomme", "Banane", "Orange"];

// Indice :       0         1         2
console.log(fruits[0]); // Pomme
console.log(fruits[1]); // Banane
console.log(fruits[2]); // Orange
```

Pour accéder à un élément, on écrit le nom du tableau suivi de crochets et de l'indice. Si l'indice demandé n'existe pas, JavaScript renvoie généralement **undefined**.

```
console.log(fruits[3]); // undefined
```

3. Créer un tableau

La manière la plus courante consiste à utiliser les crochets **[ ]**. On peut créer un tableau déjà rempli ou un tableau vide.

```
let couleurs = ["rouge", "vert", "bleu"];
let utilisateurs = [];
let notes = [12, 15, 18];
```

Il existe aussi `new Array()`, mais cette écriture peut surprendre lorsqu'on lui donne un seul nombre : ce nombre peut définir la longueur initiale du tableau, et non devenir un élément.

```
let tableauA = new Array(3);
console.log(tableauA.length); // 3
```

```
let tableauB = [3];
console.log(tableauB.length); // 1
```

Pour débuter, privilégie la notation avec les crochets `[]`.

4. Types de valeurs dans un tableau

Un tableau peut contenir des nombres, du texte, des booléens ou même plusieurs types différents. Techniquement, JavaScript l'autorise. En pratique, il est souvent plus clair de regrouper des valeurs de même nature. Les objets permettront ensuite de décrire un produit avec plusieurs propriétés.

```
let valeurs = ["Imprimante", 120, true];
```

5. Connaître la longueur : length

La propriété `length` indique le nombre d'emplacements du tableau. Elle ne s'écrit pas avec des parenthèses : c'est une propriété, pas une méthode.

```
let animaux = ["Chat", "Chien", "Lapin"];

console.log(animaux.length); // 3
```

Pour accéder au dernier élément sans connaître à l'avance la taille du tableau, on utilise `tableau[tableau.length - 1]`. On retire 1 car les indices commencent à zéro.

```
console.log(animaux[animaux.length - 1]); // Lapin
```

6. Modifier un élément

On peut remplacer une valeur en affectant une nouvelle valeur à son indice.

```
let materiel = ["Clavier", "Souris", "Écran"];

materiel[1] = "Webcam";

console.log(materiel);
// ["Clavier", "Webcam", "Écran"]
```

L'indice 1 désigne le deuxième élément. L'affectation remplace l'ancienne valeur, elle n'ajoute pas un élément à la fin.

Évite de créer volontairement des trous en écrivant à un indice très éloigné. Cela peut produire un tableau avec des emplacements vides.

7. Ajouter des éléments : push() et unshift()

push() ajoute un ou plusieurs éléments à la fin du tableau. La méthode renvoie la nouvelle longueur.

```
let fruits = ["Pomme", "Banane"];

let nouvelleLongueur = fruits.push("Orange", "Kiwi");

console.log(fruits);           // ["Pomme", "Banane", "Orange", "Kiwi"]
console.log(nouvelleLongueur); // 4
```

unshift() ajoute un ou plusieurs éléments au début du tableau. Les anciens éléments se décalent vers la droite.

```
let fruits = ["Banane", "Orange"];

fruits.unshift("Pomme");

console.log(fruits); // ["Pomme", "Banane", "Orange"]
```

8. Supprimer des éléments : pop() et shift()

pop() supprime le dernier élément et renvoie la valeur supprimée.

```
let outils = ["Clavier", "Souris", "Écran"];

let elementSupprime = outils.pop();

console.log(elementSupprime); // Écran
console.log(outils);          // ["Clavier", "Souris"]
```

shift() supprime le premier élément et renvoie la valeur supprimée.

```
let outils = ["Clavier", "Souris", "Écran"];

let premier = outils.shift();

console.log(premier); // Clavier
console.log(outils);  // ["Souris", "Écran"]
```

À retenir : push/unshift ajoutent ; pop/shift retirent. Les quatre méthodes modifient le tableau d'origine.

9. Parcourir un tableau avec for

Une boucle permet de traiter chaque élément sans écrire une instruction séparée pour chaque indice.

```
let notes = [12, 15, 8];

for (let i = 0; i < notes.length; i++) {
    console.log(notes[i]);
}
```

Explication ligne par ligne :

• `let i = 0` : on commence au premier indice.

• `i < notes.length` : on continue tant que l'indice est inférieur au nombre d'éléments.

• `i++` : on augmente l'indice de 1 après chaque passage.

• `notes[i]` : on lit l'élément situé à l'indice actuel.

On utilise `i < notes.length`, et non `i <= notes.length`, car le dernier indice valide est length - 1.

10. Parcourir avec for...of

Lorsque l'indice ne nous intéresse pas, `for...of` permet de lire directement les valeurs.

```
let fruits = ["Pomme", "Banane", "Orange"];

for (let fruit of fruits) {
    console.log(fruit);
}
```

À chaque tour, la variable `fruit` reçoit l'élément suivant. Cette forme est souvent plus simple à lire. La boucle classique `for` reste utile lorsqu'on a besoin de l'indice.

11. Vérifier la présence d'une valeur : includes()

La méthode `includes()` répond par `true` si une valeur est présente et par `false` sinon.

```
let fruits = ["Pomme", "Banane", "Orange"];

console.log(fruits.includes("Banane")); // true
console.log(fruits.includes("Kiwi"));   // false
```

Pour les chaînes de caractères, la comparaison tient compte des majuscules et minuscules : `"pomme"` et `"Pomme"` sont différentes.

12. Reconnaître un tableau

En JavaScript, `typeof` appliqué à un tableau renvoie `"object"`. Ce n'est donc pas le bon outil pour vérifier si une valeur est un tableau.

```
let liste = ["A", "B"];

console.log(typeof liste);          // "object"
console.log(Array.isArray(liste));  // true
console.log(Array.isArray("texte"));// false
```

Utilise `Array.isArray()` pour tester correctement.

13. Référence et copie simple

Quand on affecte un tableau à une autre variable, on ne crée pas automatiquement une nouvelle liste indépendante. Les deux variables peuvent désigner le même tableau.

```
let original = ["A", "B"];
let autreNom = original;

autreNom.push("C");

console.log(original); // ["A", "B", "C"]
```

Pour créer une copie superficielle d'un tableau simple, on peut utiliser l'opérateur de décomposition (spread) avec des crochets.

```
let original = ["A", "B"];
let copie = [...original];

copie.push("C");

console.log(original); // ["A", "B"]
console.log(copie);    // ["A", "B", "C"]
```

Attention : cette copie est superficielle. Si le tableau contient des objets ou d'autres tableaux, les éléments imbriqués ne sont pas automatiquement copiés en profondeur.

14. Fiche mémo

Opération	Syntaxe	Effet
Créer	let liste = []	Crée un tableau vide
Lire	liste[0]	Lit le premier élément
Longueur	liste.length	Donne le nombre d'éléments
Modifier	liste[1] = valeur	Remplace un élément
Ajouter à la fin	liste.push(valeur)	Ajoute à la fin
Ajouter au début	liste.unshift(valeur)	Ajoute au début
Retirer à la fin	liste.pop()	Retire et renvoie le dernier
Retirer au début	liste.shift()	Retire et renvoie le premier
Tester	liste.includes(valeur)	Renvoie true ou false
Vérifier le type	Array.isArray(liste)	Vérifie si c'est un tableau

15. Exercices d'entraînement

Essaie de résoudre les exercices sans regarder une correction toute faite. Écris le code dans ton éditeur, exécute-le, puis observe le résultat.

Exercice 1 – Créer et lire une liste d'étudiants

• Crée un tableau nommé `etudiants` contenant cinq prénoms.

• Affiche le tableau complet.

• Affiche le premier étudiant et le troisième étudiant.

• Affiche le nombre total d'étudiants.

• Affiche le dernier étudiant en utilisant `length - 1`, sans écrire directement son indice.

Exercice 2 – Modifier une liste de produits

```
let produits = ["Clavier", "Souris", "Écran"];
```

• Remplace « Souris » par « Webcam » en utilisant son indice.

• Ajoute « Imprimante » à la fin.

• Ajoute « Unité centrale » au début.

• Supprime le dernier élément.

• Affiche le tableau final et sa longueur.

Exercice 3 – Parcourir des notes

```
let notes = [12, 15, 8, 17, 10, 19];
```

• Affiche chaque note avec une boucle `for`.

• Refais l'affichage avec `for...of`.

• Affiche uniquement les notes supérieures ou égales à 10.

• Compte le nombre de notes supérieures ou égales à 10.

Exercice 4 – Gérer un stock

```
let stock = ["Ordinateur", "Clavier", "Souris", "Écran", "Imprimante"];
```

• Affiche tous les produits.

• Ajoute « Disque dur ».

• Vérifie si « Souris » est dans le tableau avec `includes()`.

• Supprime le dernier élément.

• Affiche le nombre d'éléments puis le tableau final.

Exercice 5 – Petit défi : prix

```
let prix = [500, 30, 15, 200, 80];
```

• Calcule la somme de tous les prix avec une boucle.

• Compte les prix strictement supérieurs à 50.

• Calcule la moyenne des prix (somme divisée par nombre de valeurs).

16. QCM de vérification

1. Quel est l'indice du premier élément ?
A. 1 B. 0 C. -1

2. Quelle propriété donne le nombre d'éléments ?
A. size B. count() C. length

3. Quelle méthode ajoute à la fin ?
A. push() B. pop() C. shift()

4. Quelle méthode retire le premier élément ?
A. pop() B. shift() C. unshift()

5. Quelle méthode vérifie la présence d'une valeur ?
A. includes() B. contains() C. exists()

6. Quelle méthode vérifie qu'une valeur est un tableau ?
A. typeof() B. isArray() C. Array.isArray()

Réponses du QCM

1 : B • 2 : C • 3 : A • 4 : B • 5 : A • 6 : C

17. Auto-évaluation : suis-je prêt à passer au Jour 9 ?

• Je sais créer un tableau avec des crochets.

• Je sais expliquer pourquoi le premier indice est 0.

• Je sais lire, modifier et compter les éléments.

• Je sais ajouter et supprimer au début ou à la fin.

• Je sais parcourir un tableau avec for et for...of.

• Je sais utiliser includes() et Array.isArray().

• Je comprends la différence entre deux variables qui partagent le même tableau et une copie avec spread.

Règle de progression

Ne passe pas au Jour 9 tant que tu ne peux pas refaire les exercices 1 et 2 sans aide, puis expliquer chaque ligne de ton code. Le but est de comprendre, pas seulement de recopier.